UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2

SECCIÓN N

SEGUNDO SEMESTRE 2026



Ensayo Proyecto 03

Nombre completo:	Carné:
Angel Eduardo Balcárcel Calderón	202405840

23 de octubre de 2026

Índice

Índice
Introducción
Arquitectura de la Solución y Paradigma POO
Implementación de Algoritmos y Estructuras Propias
Persistencia de Datos
Conclusiones
Anexos

Introducción

El presente documento detalla el diseño e implementación del Proyecto 3 para el curso de Introducción a la Programación y Computación 2. El objetivo principal fue desarrollar un sistema que simule la gestión de Documentos Tributarios Electrónicos (DTE) para la Superintendencia de Administración Tributaria (SAT). Para ello, se construyó una solución integral que separa las responsabilidades en dos componentes principales: un Frontend desarrollado en Razor Pages para la interacción con el usuario, y un Backend robusto implementado como una API RESTful en .NET 8, el cual procesa la lógica de negocio y gestiona la persistencia de los datos. La solución hace énfasis en el uso de la Programación Orientada a Objetos (POO), la validación algorítmica de identidades tributarias (NIT) y el procesamiento de texto mediante expresiones regulares.

Arquitectura de la Solución y Paradigma POO

El sistema fue diseñado bajo una arquitectura de N-Capas, garantizando la separación de responsabilidades (SoC).

- Capa de Presentación (Frontend):
  Desarrollada con Razor Pages, esta aplicación corre en el puerto 5001. Su función es capturar las solicitudes del usuario (como la carga de archivos JSON) y consumir los servicios del backend mediante peticiones HTTP asíncronas a través de HttpClient, mostrando los resultados de forma amigable (tablas, gráficas y reportes).

- Capa de Negocio (Backend/API):
  Construida con .NET Core, expone endpoints HTTP en el puerto 5000. Esta API es capaz de ser consumida no solo por el frontend, sino por cualquier cliente externo (como Postman), cumpliendo con los estándares RESTful. Aquí residen los Controladores, que reciben las peticiones, y los Servicios (como RtuService y DteService), que contienen las reglas del negocio.

- Capa de Datos:
  Implementada a través de un servicio de persistencia (DatabaseService) que interactúa con archivos planos en formato JSON, almacenando el Registro Tributario Unificado (RTU), el historial de facturas y los correlativos diarios.

El código fue estructurado aplicando los pilares de la Programación Orientada a Objetos (POO). Se utilizó la Abstracción para crear modelos de datos (como las clases Factura y Contribuyente). Se aplicó el Encapsulamiento manteniendo la lógica de lectura y escritura de archivos oculta dentro de la clase DatabaseService, impidiendo que otras clases manipulen los archivos directamente. Finalmente, la Herencia y el Polimorfismo se reflejan en el uso de interfaces implícitas y clases de servicio desacopladas.

Implementación de Algoritmos y Estructuras Propias

Uno de los mayores retos técnicos del proyecto fue la correcta implementación de las validaciones y el uso de Tipos de Datos Abstractos (TDA).

- Uso de TDA Propio (Cola Enlazada):
  Para cumplir con el requerimiento de construir estructuras desde cero, se implementó el TDA LinkedQueue<T>. Esta es una cola FIFO (First-In, First-Out) basada en nodos enlazados. Se utilizó dentro del DteService para encolar las facturas recibidas antes de procesarlas. Esto garantiza que, si llega un lote de 100 facturas, se procesen en el estricto orden en que fueron recibidas, respetando la naturaleza secuencial del correlativo de autorización.

- Validación del NIT (Módulo 11):
  Se desarrolló la clase estática NitValidator. El algoritmo recorre el NIT de la cadena de menor peso a mayor peso (excluyendo el dígito verificador), multiplicando cada número por su posición. El resultado de la sumatoria se le aplica el módulo 11. El resultado se resta de 11 y se vuelve a aplicar módulo 11. Si el resultado es 10, se asigna la letra "K". Esta lógica asegura que no se autoricen facturas a contribuyentes con NITs falsos.

- Expresiones Regulares:
  Para el campo "TIEMPO" (ej. "Guatemala, 15/01/2025 15:25 hrs."), se implementó un extractor basado en expresiones regulares en la clase RegexExtractor. Mediante el patrón (\d{1,2}/\d{1,2}/\d{4}\s+\d{1,2}:\d{2}), el sistema ignora el texto circundante (como el nombre de la ciudad o la abreviatura de horas) y extrae exclusivamente la fecha. Posteriormente, se utiliza DateTime.TryParseExact con múltiples formatos (para aceptar días de uno o dos dígitos) para validar que la fecha sea real (por ejemplo, rechazar un 29 de febrero en un año no bisiesto).

- Manejo de Precisión Financiera:
  Para evitar errores de redondeo de punto flotante, se utilizó el tipo de dato decimal en C#. Para el cálculo del IVA, se implementó la fórmula Math.Round(valor * 0.12m, 2, MidpointRounding.AwayFromZero), la cual asegura que siempre se redondee hacia arriba en los casos de empate (como exige el sistema financiero).

Persistencia de Datos

La persistencia de datos se gestionó mediante el uso de archivos JSON. Se diseñó un servicio DatabaseService que utiliza System.Text.Json para serializar y deserializar las listas de objetos. Para evitar condiciones de carrera (Race Conditions) durante la escritura simultánea, se implementó el uso de hilos y candados (lock) en la asignación de correlativos diarios, guardando un diccionario con la fecha como llave y el último correlativo asignado como valor.

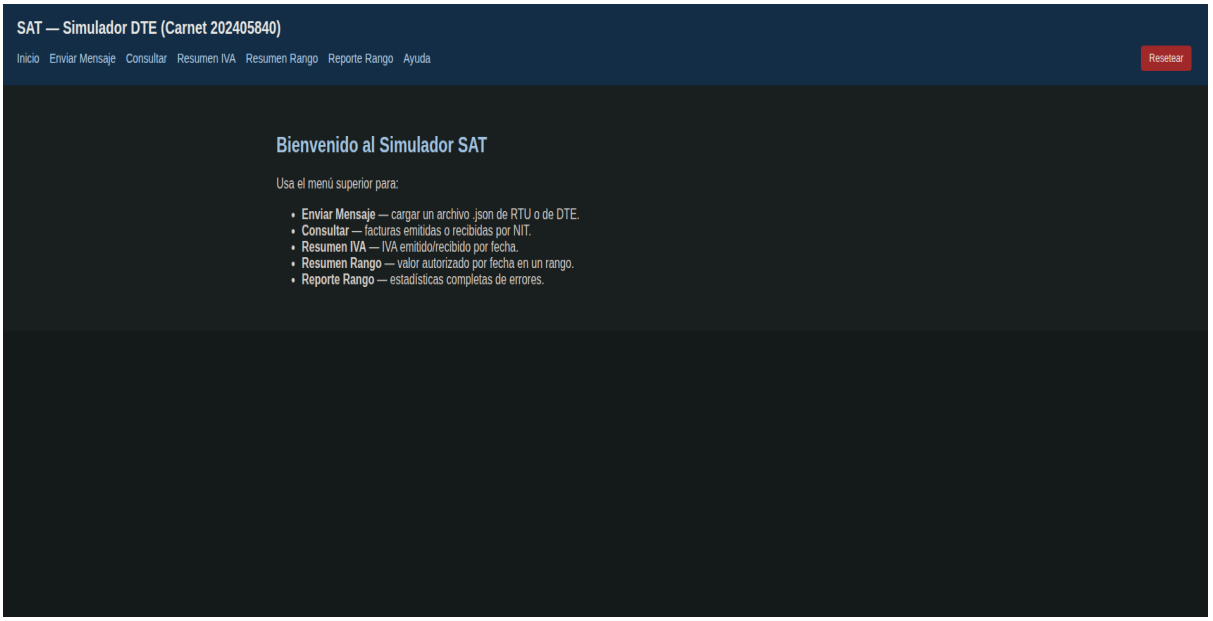
El sistema clasifica las facturas en tres estados:

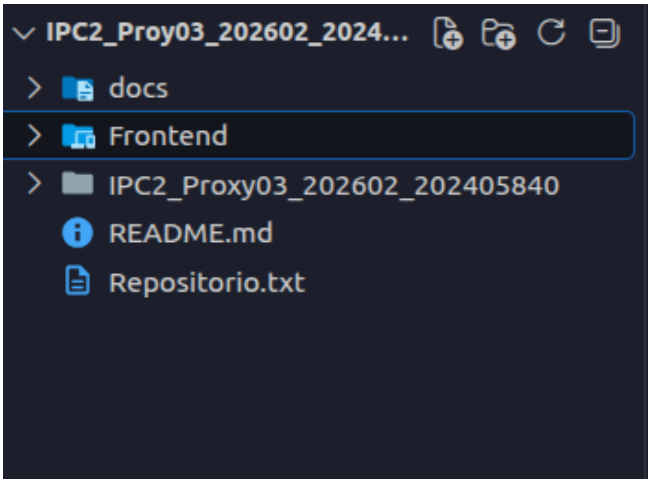
- Aprobadas (cumplen todas las reglas)
- Rechazadas (tienen errores de negocio como IVA incorrecto
- NIT inexistente o referencias duplicadas) y Descartadas por Formato (el texto de fecha no pudo ser interpretado por las expresiones regulares). Esta distinción es crucial para generar el reporte estadístico en el frontend.

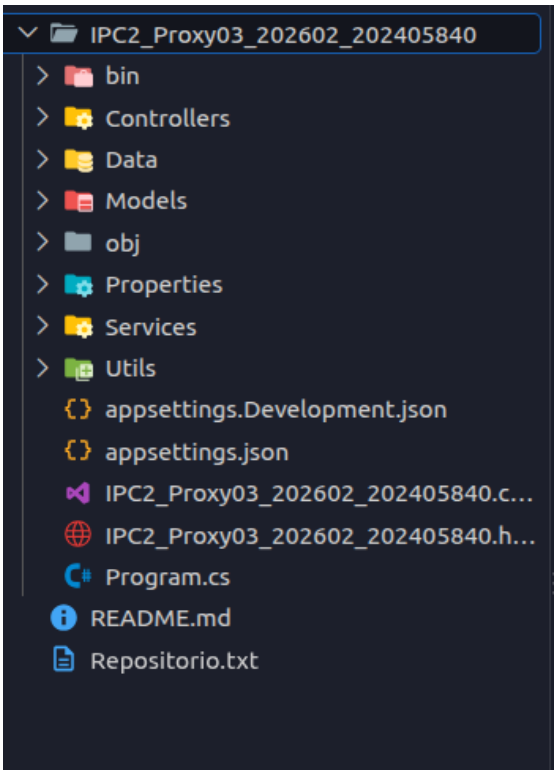
Conclusiones

El desarrollo de este proyecto permitió consolidar los conocimientos en el desarrollo web moderno con .NET. La correcta aplicación de la programación orientada a objetos no solo facilitó la escritura del código, sino que hizo que el mantenimiento y la depuración (como el tratamiento de JSONs malformados) fueran tareas sencillas. La implementación de estructuras de datos propias demostró la importancia de entender cómo funcionan los cimientos de la programación, y el uso de expresiones regulares se reveló como la herramienta óptima para el análisis de datos no estructurados en entornos reales.

Anexos


SAT — Simulador DTE (Carnet 202405840)
Inicio
Enviar Mensaje
Consultar
Resumen IVA
Resumen Rango
Reporte Rango
Ayuda
Resetear
Bienvenido al Simulador SAT
Usa el menú superior para:
Enviar Mensaje — cargar un archivo .json de RTU o de DTE.
Consultar — facturas emitidas o recibidas por NIT.
Resumen IVA — IVA emitido/recibido por fecha.
Resumen Rango — valor autorizado por fecha en un rango.
Reporte Rango — estadísticas completas de errores.


IPC2_Proy03_202602_2024...
docs
Frontend
IPC2_Proxy03_202602_202405840
README.md
Repositorio.txt


IPC2_Proxy03_202602_202405840
bin
Controllers
Data
Models
obj
Properties
Services
Utils
appsettings.Development.json
appsettings.json
IPC2_Proxy03_202602_202405840.c...
IPC2_Proxy03_202602_202405840.h...
Program.cs
README.md
Repositorio.txt